

# RESIDUAL-ENHANCED ADAPTIVE KOOPMAN AUTOENCODER: A DEEP LATENT DYNAMICS MODEL FOR STOCK PREDICTION

Lei Liao<sup>1</sup>   Yang Zhang<sup>2</sup>   Jun Wang<sup>2</sup>   Jinghua Tan<sup>3\*</sup>   Yinchao Liao<sup>1</sup>

<sup>1</sup> School of Finance, Southwestern Univ. of Finance and Economics

<sup>2</sup> School of Management Science and Engineering, Southwestern Univ. of Finance and Economics

<sup>3</sup> College of Economics, Sichuan Agricultural Univ.

## ABSTRACT

Stock markets are complex dynamical systems shaped by macro fundamentals, policy, and investor behavior. Their evolution is non-stationary with recurrent market states and abrupt shocks, which challenges models that assume stable distributions. We tackle this problem with modern Koopman theory, which portrays complex dynamical systems and considers such nonstationary dynamics. Specifically, we propose REAKA, a Residual-Enhanced Adaptive Koopman Autoencoder for modeling stock-return dynamics in the latent space, with a residual path to capture higher-order nonlinearities. REAKA introduces an Adaptive Koopman Selector to adaptively choose the appropriate operator for different market conditions and a diffusion-based residual corrector to handle noise, abrupt shocks, and nonlinear effects beyond the limits of finite-dimensional Koopman linearization. By embedding these modules within an autoencoder, REAKA learns Koopman-invariant coordinates and advances dynamics in latent space. Experiments on real stock market data demonstrate that REAKA outperforms existing methods, significantly improving prediction accuracy and robustness in complex financial environments.

**Index Terms**— Stock Prediction, Koopman Theory, Diffusion model, Autoencoder

## 1. INTRODUCTION

In financial markets, the evolution of the price of stocks is a complex nonlinear dynamical system driven by coupled macro conditions, market microstructure, and investor behavior [1]. The interplay of these factors leads to complex dynamical characteristics in stock price series, including nonstationarity, clustering of volatility, and frequent change in market state [2, 3]. Although traditional time series models such as ARIMA are interpretable, their linear assumption severely limits their ability to capture nonlinear market dynamics [4]. Recent deep learning models such as LSTMs and Transformers improve end-to-end fitting of nonlinear dynamics [5]. However, these advances predominantly address nonstationarity through the perspective of architectural design of models, rather than through a fundamental theoretical framework for modeling financial market states shifts.

From a dynamical system perspective, Koopman theory provides a principled foundation for modeling complex nonlinear dynamical systems by lifting the state into an observable (measurement) function space in which the evolution is linear under a Koopman operator [6]. It has been applied in fluid dynamics, engineering systems, and biological networks, among others [7, 8]. To han-

dle time-varying dynamics, researchers have proposed partition-wise linearization with localized Koopman to represent different dynamical mechanisms [9]. This approach is especially suitable for multi-state financial systems.

However, existing Koopman approaches still face challenges in financial forecasting. First, many studies employ fixed operators, which cannot adequately capture the dynamical differences among multiple market states. Second, finite-dimensional approximations inevitably leave a residual term. This residual is not white noise and contains higher-order nonlinear structure not captured by linear evolution and is typically state dependent, non-Gaussian, and shock-prone [10, 11]. Directly fitting such residuals with standard neural networks (e.g., MLP) tends to underestimate extremes, since the problem is to learn a complex conditional distribution rather than a point estimate. Diffusion models are better suited here, as they provide a probabilistic framework to learn complex conditional distributions and time-varying uncertainty [12].

To address these issues, this paper proposes a **Residual-Enhanced Adaptive Koopman Autoencoder (REAKA)** that combines multiple Koopman operators with a residual correction module from a dynamical systems perspective. Specifically, the model introduces multiple learnable Koopman operators, enabling adaptive selection and dynamic switching among market mechanisms at different periods, thus improving adaptability to market structure changes and regime shifts. Additionally, a novel diffusion-based residual correction module explicitly models complex, noise, and high-order nonlinear in the latent space, effectively capturing intricate market dynamics that conventional Koopman modeling may overlook. Empirical results on real-world financial data demonstrate that the proposed method significantly outperforms the mainstream deep learning and single Koopman operator models in terms of prediction accuracy, robustness, and metrics such as RankIC and RankICIR.

## 2. PRELIMINARIES

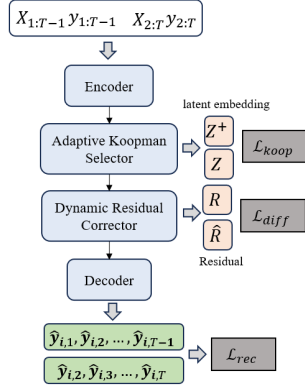
### 2.1. Problem Formulation

Given a collection of  $S$  financial assets, let  $X_i = [x_{i,1}, x_{i,2}, \dots, x_{i,T}]$  denote the historical feature sequence for asset  $i$ , where  $x_{i,t} \in \mathbb{R}^F$  is the  $F$ -dimensional feature vector at time  $t$ , and let  $y_i = [y_{i,1}, y_{i,2}, \dots, y_{i,T}]$  be the corresponding historical returns. The objective is to learn a function  $\mathcal{M}$  to predict the future return(s) of each asset at time  $t + 1$ :  $\hat{y}_{i,t+1} = \mathcal{M}(X_i^{1:t}, y_i^{1:t})$ , where  $\hat{y}_{i,t+1}$  is the predicted return,  $X_i^{1:t} = [x_{i,1}, \dots, x_{i,t}]$ , and  $y_i^{1:t} = [y_{i,1}, \dots, y_{i,t}]$ .

Email: jinghuatan@sicau.edu.cn. \*Corresponding author.

### 3. METHODOLOGY

As shown in Figure 1, our model follows an autoencoder framework with four parts. First, a dual-path LSTM encoder maps historical prices and auxiliary stock features to latent representations, and a gating module fuses them into a single latent state. Second, an adaptive selector chooses the most appropriate operator according to the current latent state to effectively capture market state transitions in financial markets. Third, a diffusion-based residual corrector predicts components not explained by the Koopman operator to handle nonlinear effects and idiosyncratic shocks. Fourth, a decoder maps latent states back to the state space for reconstruction and forecasting. All modules are trained end-to-end with three losses: reconstruction, Koopman consistency, and residual prediction. At inference, only past returns and features are used.



**Fig. 1.** Overview of the Residual-Enhanced Adaptive Koopman Autoencoder architecture.

#### 3.1. Latent State Encoder

We encode the observed stock return sequences into a latent space via autoencoder. It aims to approximate the measurement functions, thereby facilitating the modeling of temporal dynamics as linear state transitions in the latent space. Specifically, inspired by the Dynamic Mode Decomposition (DMD) algorithm [13], we divide the stock return sequence  $\mathbf{y} \in \mathbb{R}^{T \times 1}$  into two overlapping segments, denoted as  $\mathbf{y}_{1:T-1}$  and  $\mathbf{y}_{2:T}$ .

$$\mathbf{y}_{1:T-1} = [y_1, y_2, \dots, y_{T-1}] \quad (1)$$

$$\mathbf{y}_{2:T} = [y_2, y_3, \dots, y_T] \quad (2)$$

For stock return time series, obtaining faithful observable coordinates typically requires a time-delay embedding [14]. Instead of constructing explicit Hankel delay vectors, we adopt an implicit delay embedding. We use an LSTM encoder that outputs latent embeddings at every time step, and stacking per-step latent embeddings across time yields snapshot matrices  $H_y$  and  $H_y^+$  [15].

$$H_y = \text{LSTM}_y(\mathbf{y}_{1:T-1}) \quad (3)$$

$$H_y^+ = \text{LSTM}_y(\mathbf{y}_{2:T}) \quad (4)$$

We incorporate stock-specific features into the latent encoding, since returns depend on market and fundamental signals beyond their own history. An additional LSTM encodes the feature sequences  $\mathbf{x}_{1:T-1}$  and  $\mathbf{x}_{2:T}$ .

$$H_x = \text{LSTM}_x(\mathbf{x}_{1:T-1}) \quad (5)$$

$$H_x^+ = \text{LSTM}_x(\mathbf{x}_{2:T}) \quad (6)$$

To effectively integrate both sources of information, a gating mechanism adaptively fuses the latent representations derived from return and feature sequences.

$$G = \text{GateNet}(H_x) \quad (7)$$

$$G^+ = \text{GateNet}(H_x^+) \quad (8)$$

$$Z = H_y \odot G + H_x \odot (1 - G) \quad (9)$$

$$Z^+ = H_y^+ \odot G^+ + H_x^+ \odot (1 - G^+) \quad (10)$$

Here,  $\text{GateNet}(\cdot)$  is a multilayer perceptron followed by a sigmoid, which generates a gating vector  $G \in [0, 1]$  to adaptively weigh each source;  $\odot$  denotes element-wise multiplication.

Finally, we learn latent embedding  $Z$  and  $Z^+$  in latent space such that the system evolution can be approximated by Koopman operators  $\mathcal{K}$ , the latent space acts as an approximate invariant subspace under the selected operator. This aligns with the Koopman theory concept of invariant subspaces, enabling the linear evolution of complex dynamics within the learned space.

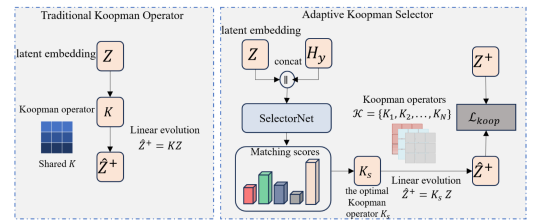
#### 3.2. Adaptive Koopman Selector

Building on Koopman operator theory, we model the nonlinear evolution of financial returns by mapping them into a latent space. According to the Koopman framework, the system evolves as  $\phi(\mathbf{y}_{i,t+1}) = (\mathcal{K}\phi)(\mathbf{y}_{i,t})$  where  $\phi(\cdot)$  denotes the measurement function and  $\mathcal{K}$  is the Koopman operator. In our implementation, this relationship is approximated by a learnable matrix  $K$  in the latent space, so the evolution becomes  $\hat{Z}^+ = KZ$ , where  $\hat{Z}^+$  is the predicted latent state and  $Z$  is the current latent state. Training minimizes the discrepancy  $\|\hat{Z}^+ - Z^+\|$  to capture the linearized dynamical structure. A single global Koopman operator  $K$  assumes that the entire latent evolution is captured by a fixed linear mapping. To overcome this limitation, we propose the Adaptive Koopman Selector (AKS) with a codebook  $\mathcal{K} = \{K_1, K_2, \dots, K_N\}$ . At each time step, a selector network produces matching scores based on the latent state  $Z$  and the initial latent representation  $H_y$ .

$$\mathbf{a} = \text{SelectorNet}(Z, H_y), \quad (11)$$

which, for simplicity, is a single-layer MLP with LeakyReLU [16]:

$$\mathbf{a} = \text{LeakyReLU}(W[Z, H_y]\mathbf{h}). \quad (12)$$



**Fig. 2.** Traditional vs. Adaptive Koopman Operator Selector.

To obtain a differentiable yet discrete selection, we use the Gumbel-Softmax trick [17]. Let  $\epsilon_i \sim \text{Gumbel}(0, 1)$  and  $\tau > 0$  be the temperature. The relaxed one-hot weights are

$$\alpha_i = \frac{\exp((a_i + \epsilon_i)/\tau)}{\sum_{j=1}^N \exp((a_j + \epsilon_j)/\tau)}, \quad i = 1, \dots, N. \quad (13)$$

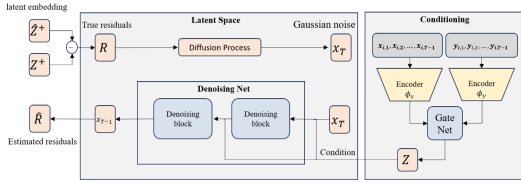
The selected (or softly mixed) operator and the latent evolution are then

$$K_s = \sum_{i=1}^N \alpha_i K_i, \quad (14)$$

$$\hat{Z}^+ = K_s Z. \quad (15)$$

### 3.3. Dynamic Residual Corrector

While the Koopman operator captures the primary linear dynamics via finite-dimensional approximations, real-world financial series exhibit complex nonlinear behaviors such as noise, abrupt shocks, and higher-order effects that are not fully explained by linear modeling. Let the residual be  $R = Z^+ - \hat{Z}^+$ , where  $Z^+$  is the true next-step latent state and  $\hat{Z}^+$  is the state advanced by Koopman evolution. The magnitude and structure of  $R$  vary across market states and reflect nonlinear processes rather than pure noise. To address this limitation, we design a Dynamic Residual Corrector based on a conditional diffusion model that learns to generate context-aware residuals given the current latent state  $Z$ . Diffusion models progressively transform noise into data through iterative denoising, which allows them to learn complex and changing residual patterns [12]. Conditioning on  $Z$  enables the corrector to produce residuals that reflect the current market environment.



**Fig. 3.** Architecture of the Dynamic Residual Corrector based on Conditional Diffusion.

During training we follow the standard DDPM forward process with a noise schedule  $\{\beta_t\}_{t=1}^T$ , where  $\alpha_t = 1 - \beta_t$  and  $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ :

$$x_t = \sqrt{\bar{\alpha}_t} R + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (16)$$

The denoiser  $\epsilon_\theta(x_t, t, Z)$  is conditioned on  $Z$  and trained with the standard noise-prediction objective

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{t, \epsilon} \|\epsilon_\theta(x_t, t, Z) - \epsilon\|_2^2. \quad (17)$$

At inference, we start from  $x_T \sim \mathcal{N}(0, I)$  and apply the reverse updates for  $t = T, \dots, 1$ :

$$\mu_\theta(x_t, t, Z) = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t, Z) \right), \quad (18)$$

$$\sigma_t^2 = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t, \quad (19)$$

$$x_{t-1} = \mu_\theta(x_t, t, Z) + \sigma_t \xi, \quad \xi \sim \mathcal{N}(0, I). \quad (20)$$

After the final step we map  $x_0$  to a residual sample  $\hat{R}$ . The corrected latent state at the next time step is then computed as:

$$\tilde{Z}^+ = \hat{Z}^+ + \hat{R}. \quad (21)$$

The refined latent  $\tilde{Z}^+$  is then fed into the decoder to reconstruct or forecast the target output. All components are trained jointly so that both linear and nonlinear patterns are captured effectively.

### 3.4. Decoder

The decoder is responsible for reconstructing both the historical and predicted return sequences from the latent representations. Specifically, the decoder takes as input the corrected latent states corresponding to the historical segment.

$$\mathbf{y}_{1:T-1} = \text{Decoder}(Z), \quad (22)$$

$$\mathbf{y}_{2:T} = \text{Decoder}(\tilde{Z}^+), \quad (23)$$

where  $Z$  denotes the latent representations for the historical sequence, and  $\tilde{Z}^+$  is the refined latent state after residual correction. This reconstruction ensures that the learned latent space preserves the key information from both the input and predicted time steps, thereby enhancing model robustness.

### 3.5. Predictor

During inference, the trained model takes as input the historical return sequence  $y_{1:t}$  and the corresponding feature sequence  $x_{1:t}$ , which are encoded into latent representations via encoder, resulting in the fused latent state  $Z$ . The model then applies the Adaptive Koopman Selector and the Dynamic Residual Corrector to update the latent state to  $\tilde{Z}^+$ . The latent state is updated as:

$$K_s = \arg \max \text{SelectorNet}(Z, H_y), \quad (24)$$

$$\hat{Z}^+ = K_s Z, \quad (25)$$

$$\tilde{Z}^+ = \hat{Z}^+ + \text{ResidualNet}(Z), \quad (26)$$

$$\hat{\mathbf{y}}_{2:t+1} = \text{Decoder}(Z^+). \quad (27)$$

Therefore, the estimated return  $\hat{y}_{t+1}$  is given by the last element of the reconstructed sequence  $\hat{\mathbf{y}}_{2:t+1}$ .

### 3.6. Loss function

The loss function includes the reconstruction loss for both the historical return sequence and the predicted future return, ensuring that the latent space preserves essential information; the Koopman consistency loss, encourages the linearly evolved latent state to align with the true latent representation in the next time step; the residual consistency loss, ensures that the residual corrector accurately models the nonlinear residual dynamics. The total loss is given by:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{koop}} + \mathcal{L}_{\text{diff}}, \quad (28)$$

where

$$\mathcal{L}_{\text{rec}} = \text{MSE}(\hat{\mathbf{y}}_{1:T-1}, \mathbf{y}_{1:T-1}) + \text{MSE}(\hat{\mathbf{y}}_{2:T}, \mathbf{y}_{2:T}), \quad (29)$$

$$\mathcal{L}_{\text{koop}} = \text{MSE}(\hat{Z}^+, Z^+), \quad (30)$$

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{t, \epsilon} [\|\epsilon_\theta(x_t, t, Z) - \epsilon\|_2^2]. \quad (31)$$

## 4. EXPERIMENT

### 4.1. Experiment Setup

We conduct empirical evaluations using two representative stock market datasets: the CSI300 and the S&P500, which cover both the Chinese and US equity markets. The sample period ranges from January 1, 2010 to December 31, 2020. We utilize the Alpha158 dataset from the Qlib platform [18], which provides 158 technical features. The lookback window length  $T$  is set to 10 trading days. The data is split into training (January 1, 2010 to December 31, 2017), validation (January 1, 2018 to December 31, 2018), and test (January 1, 2019 to December 31, 2020) sets. We adopt the rank information coefficient (RankIC) and Information Ratio (RankICIR), defined as  $\text{RankIC}_t = \text{Spearman}(\text{rank}(\hat{\mathbf{r}}_t), \text{rank}(\mathbf{r}_t))$  and  $\text{RankICIR} = \text{mean}(\text{RankIC}_t) / \text{mean}(\text{RankIC}_t)$ , where  $\hat{\mathbf{r}}_t$  and  $\mathbf{r}_t$  denote predicted and realized returns.

### 4.2. Baseline

We compare REAKA with eight widely used baselines: LightGBM [19], recurrent models LSTM and GRU for temporal de-

**Table 1.** Overall performance comparison.

| Datasets  | CSI300       |              | S&P 500      |              |
|-----------|--------------|--------------|--------------|--------------|
|           | RankIC       | RankICIR     | RankIC       | RankICIR     |
| LightGBM  | 0.016        | 0.148        | 0.013        | 0.110        |
| LSTM      | 0.027        | 0.221        | 0.018        | 0.201        |
| GRU       | 0.028        | 0.239        | 0.021        | 0.212        |
| ALSTM     | 0.034        | 0.340        | 0.026        | 0.258        |
| TCN       | 0.039        | 0.334        | 0.029        | 0.306        |
| ADGATs    | 0.046        | 0.414        | 0.042        | 0.387        |
| FactorVAE | 0.042        | 0.375        | 0.042        | 0.375        |
| MASTER    | 0.052        | 0.481        | 0.048        | 0.456        |
| REKA      | <b>0.064</b> | <b>0.568</b> | <b>0.061</b> | <b>0.541</b> |

dependencies [15, 20], ALSTM which adds attention to LSTM [21], the temporal convolutional network TCN [22], the graph based ADGATs that model momentum spillovers [23], FactorVAE for interpretable latent factors [24], and MASTER, a transformer that captures cross time and cross stock correlations [25].

### 4.3. Comparative Analysis of Predictive Performance

Table 1 reports results for all baselines and our model. REAKA achieves the best overall performance, with a rankIC / rankICIR of 0.064 / 0.568 on CSI300 and 0.061/0.541 on S&P500, surpassing deep sequence models (LSTM, GRU, ALSTM, TCN) and the state-of-the-art models (ADGATs, FactorVAE, MASTER). REAKA models stock returns dynamics in a Koopman-based latent space, learns multiple operators, and applies diffusion-based residual refinement. This design captures both deterministic temporal patterns and higher-order nonlinear effects with noise and abrupt shocks, yielding stable and discriminative ranking predictions.

### 4.4. Ablation study

We test five variants to assess each component: a single-operator model without the Adaptive Koopman Selector (AKS), a model without the diffusion Residual Corrector (DRC), a model without the gating mechanism (GM) that uses returns only, a vanilla autoencoder without Koopman or residual modules, and a residual-MLP that replaces the diffusion corrector. As shown in Table 2, each module contributes meaningfully. Removing AKS reduces RankICIR from 0.568 to 0.455 on CSI300 and to 0.447 on S&P500, indicating the need for state specific operators. Removing DRC yields RankICIR = 0.463 on S&P500, showing that diffusion residual modeling improves robustness to noise and nonlinear shocks. Dropping GM lowers both metrics, confirming the value of stock specific features. The vanilla autoencoder degrades most (RankIC = 0.038 on CSI300 and 0.034 on S&P500). The residual-MLP underperforms the diffusion based corrector, suggesting that conditional diffusion models state dependent residuals more effectively.

### 4.5. Investment Simulation

To further evaluate the profitability of our method in the real stock market, we conduct an investment simulation. we use a TopK strategy, at each rebalancing date, all candidates are ranked by predicted scores and only the top  $K$  are held. In our experiments,  $K = 30$ . Figure 4 plots out-of-sample cumulative returns. REAKA achieves the highest cumulative return on both datasets. Compared with the

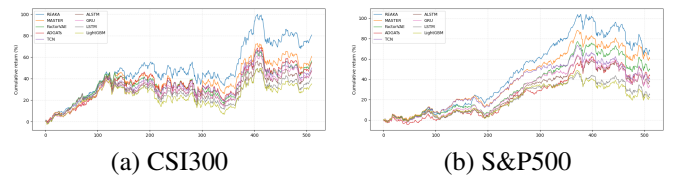
**Table 2.** Ablation study

| Datasets     | CSI300       |              | S&P 500      |              |
|--------------|--------------|--------------|--------------|--------------|
|              | RankIC       | RankICIR     | RankIC       | RankICIR     |
| w/o AKS      | 0.053        | 0.455        | 0.051        | 0.447        |
| w/o DRC      | 0.051        | 0.480        | 0.049        | 0.463        |
| w/o GM       | 0.045        | 0.421        | 0.044        | 0.419        |
| autoencoder  | 0.038        | 0.364        | 0.034        | 0.325        |
| residual-MLP | 0.052        | 0.491        | 0.050        | 0.469        |
| REKA         | <b>0.064</b> | <b>0.568</b> | <b>0.061</b> | <b>0.541</b> |

**Table 3.** Portfolio performance (AR and SR).

| Datasets  | CSI300       |             | S&P 500      |             |
|-----------|--------------|-------------|--------------|-------------|
|           | AR           | SR          | AR           | SR          |
| LightGBM  | 0.161        | 0.87        | 0.102        | 0.73        |
| LSTM      | 0.186        | 0.96        | 0.117        | 0.84        |
| GRU       | 0.201        | 1.05        | 0.151        | 1.04        |
| ALSTM     | 0.212        | 1.06        | 0.171        | 1.14        |
| TCN       | 0.221        | 1.12        | 0.181        | 1.23        |
| ADGATs    | 0.241        | 1.13        | 0.196        | 1.16        |
| FactorVAE | 0.247        | 1.18        | 0.219        | 1.45        |
| MASTER    | 0.265        | 1.25        | 0.261        | 1.67        |
| REKA      | <b>0.339</b> | <b>1.50</b> | <b>0.291</b> | <b>1.71</b> |

strongest baseline (MASTER), the final return improves by at least 11.5%. Table 3 reports annualized return (AR) and Sharpe ratio (SR). REAKA attains AR = 0.339 and SR = 1.50 on CSI300, which are improvements of 28.0% (AR) and 20.0% (SR) over MASTER. On S&P500, it achieves AR = 0.291 and SR = 1.71, improving on MASTER by 11.5% and 2.4%, respectively.

**Fig. 4.** Cumulative returns comparison on two datasets.

## 5. CONCLUSION

In this paper, we propose REAKA (Residual-Enhanced Adaptive Koopman Autoencoder), a model for stock return prediction in nonstationary markets. Specifically, REAKA follows an encoder-decoder pipeline in which a dual-path LSTM with a gating mechanism produces latent states, a codebook of learnable Koopman operators with an adaptive selector advances the states, and a diffusion-based residual corrector refines the propagated latents. Experiments on CSI300 and S&P500 show that REAKA outperforms strong baselines in RankIC and RankICIR and achieves higher cumulative returns. Ablation results verify the contribution of the selector, the residual corrector, and the gating mechanism. Future work will incorporate macroeconomic signals, model cross-asset dependencies, and explore uncertainty estimation.

## 6. ACKNOWLEDGEMENTS

This work was supported by the National Natural Science Foundation of China (NSFC) (72301188, 72471197, 72531008, 62402396), the National Social Science Foundation of China (25AZD026), the Sichuan Science and Technology Program (2025NSFSC1957), the China Postdoctoral Science Foundation (2023M732503, 2025 T181014), the Sichuan Provincial Philosophy and Social Science Fund (SCJJ25ND091), and Artificial Intelligence and Digital Finance Key Laboratory of Sichuan Province.

## 7. REFERENCES

- [1] Eugene F. Fama, "Efficient capital markets: A review of theory and empirical work," *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970.
- [2] James D. Hamilton, "A new approach to the economic analysis of nonstationary time series and the business cycle," *Econometrica: Journal of the Econometric Society*, pp. 357–384, 1989.
- [3] Shihao Gu, Bryan Kelly, and Dacheng Xiu, "Empirical asset pricing via machine learning," *The Review of Financial Studies*, vol. 33, no. 5, pp. 2223–2273, 2020.
- [4] A. A. Ariyo, A. O. Adewumi, and C. K. Ayo, "Stock price prediction using the arima model," in *2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation*. IEEE, 2014, pp. 106–112.
- [5] Qiang Li, Jing Tan, Jian Wang, and Hong Chen, "A multi-modal event-driven lstm model for stock prediction using on-line news," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 10, pp. 3323–3337, 2020.
- [6] B. O. Koopman, "Hamiltonian systems and transformation in hilbert space," *Proceedings of the National Academy of Sciences*, vol. 17, no. 5, pp. 315–318, 1931.
- [7] Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton, "Deep learning for universal linear embeddings of nonlinear dynamics," *Nature Communications*, vol. 9, no. 1, pp. 4950, 2018.
- [8] B. D. Shaffer, J. R. Vorenberg, and M. A. Hsieh, "Spectrally informed learning of fluid flows," *Chaos: An Interdisciplinary Journal of Nonlinear Science*, vol. 35, no. 3, 2025.
- [9] Joshua L. Proctor, Steven L. Brunton, and J. Nathan Kutz, "Dynamic mode decomposition with control," *arXiv preprint arXiv:1409.6358*, 2014.
- [10] Rama Cont, "Empirical properties of asset returns: stylized facts and statistical issues," *Quantitative Finance*, vol. 1, no. 2, pp. 223–236, 2001.
- [11] James B. Heaton, Nick G. Polson, and Jan Hendrik Witte, "Deep learning for finance: deep portfolios," *Applied Stochastic Models in Business and Industry*, vol. 33, no. 1, pp. 3–12, 2017.
- [12] Jonathan Ho, Ajay Jain, and Pieter Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, 2020, vol. 33, pp. 6840–6851.
- [13] D. P. Kuttichira, E. A. Gopalakrishnan, V. K. Menon, and K. P. Soman, "Stock price prediction using dynamic mode decomposition," in *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*. IEEE, 2017, pp. 55–60.
- [14] Tim Sauer, James A Yorke, and Martin Casdagli, "Embedology," *Journal of statistical Physics*, vol. 65, no. 3, pp. 579–616, 1991.
- [15] Sepp Hochreiter and Jürgen Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, pp. 1735–1780, 1997.
- [16] Bing Xu, Naiyan Wang, Tianqi Chen, and Mu Li, "Empirical evaluation of rectified activations in convolutional network," *arXiv preprint arXiv:1505.00853*, 2015.
- [17] Eric Jang, Shixiang Gu, and Ben Poole, "Categorical reparameterization with gumbel-softmax," *arXiv preprint arXiv:1611.01144*, 2016.
- [18] X. Yang, W. Liu, D. Zhou, J. Bian, and T.-Y. Liu, "Qlib: An ai-oriented quantitative investment platform," *arXiv preprint arXiv:2009.11189*, 2020.
- [19] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu, "Lightgbm: A highly efficient gradient boosting decision tree," *Advances in neural information processing systems*, vol. 30, 2017.
- [20] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.
- [21] Q. Wang and Y. Hao, "Alstm: An attention-based long short-term memory framework for knowledge base reasoning," *Neurocomputing*, vol. 399, pp. 342–351, 2020.
- [22] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.
- [23] Rui Cheng and Qing Li, "Modeling the momentum spillover effect for stock prediction via attribute-driven graph attention networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, vol. 35.
- [24] Y. Duan, L. Wang, Q. Zhang, and J. Li, "Factorvae: A probabilistic dynamic factor model based on variational autoencoder for predicting cross-sectional stock returns," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022, vol. 36, pp. 4468–4476.
- [25] T. Li, Z. Liu, Y. Shen, X. Wang, H. Chen, and S. Huang, "Master: Market-guided stock transformer for stock price forecasting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024, vol. 38, pp. 162–170.